

Study Companion — Project Review & Guide Manual

Comprehensive Technical Reference, System Architecture & Viva Preparation

1. Executive Summary & Core Value Proposition

Study Companion is an AI-powered, subject-isolated academic workspace platform designed to transform raw study materials (PDFs, DOCX files, text notes) into structured learning resources, active recall quizzes, interactive knowledge dependency graphs (DAGs), step-by-step logic flow process visualizers, and subject-bounded AI chat assistance.

Key Core Solutions:

- **Subject Isolation:** Dedicated workspaces (e.g., Operating Systems, DWDM) isolate documents and chat history, preventing AI context bleed between unrelated subjects.
- **Structured Content Generation:** Converts documents into Revision Notes, 2/5/10-mark Solved Exam Guides (with examiner coaching boxes), and Interactive MCQ Quizzes.
- **Concept Dependency Graph (DAG):** Extracts 8–20 key concepts, enforces strict prerequisites, detects and breaks circular dependencies via Depth-First Search (DFS), and tracks node mastery.
- **Active Recall ('Explain-Back') Mode:** Evaluates student explanation text against concept summaries, returning an automated score (0–100) and actionable coaching feedback.
- **Interactive Logic Flow Simulator:** Visualizes technical algorithms and multi-step processes as interactive flowcharts with step-by-step simulation controls.

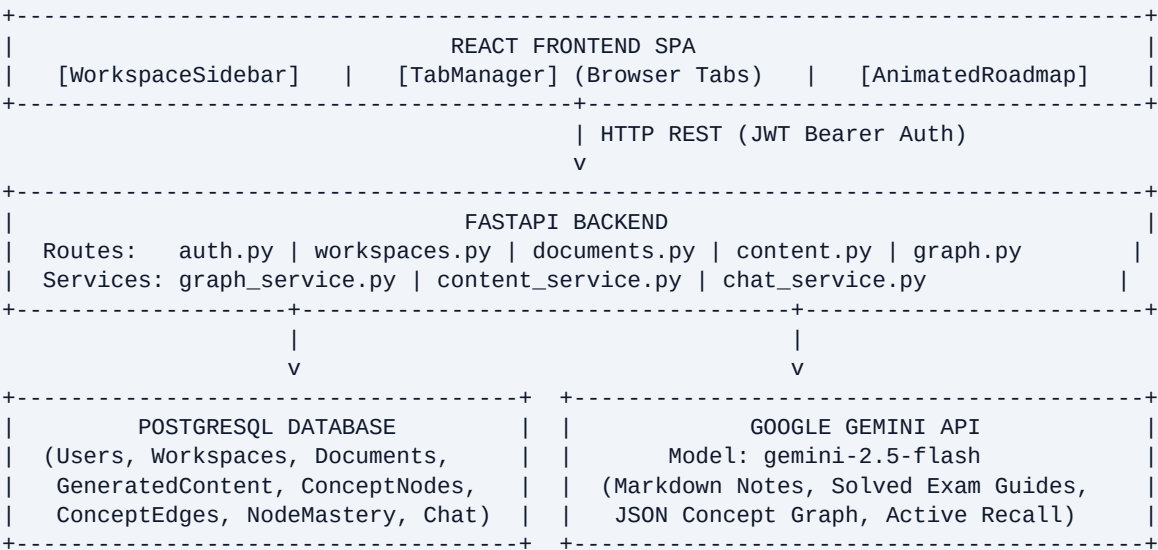
2. Technology Stack Breakdown

Layer	Technology	Role & Purpose in System
Frontend UI	React 18 + Vite	Fast component-based single-page application with browser-style multi-tab manager.
Frontend Styling	Vanilla CSS (Design System)	Custom dark/light themes, glassmorphism, dynamic animations, responsive grid/flexbox.
Icons	Lucide React	Clean vector icons (BookOpen, Map, Send, Lock, CheckCircle).
Backend API	FastAPI (Python 3.11)	High-performance async web framework with automatic Pydantic validation & OpenAPI docs.
ASGI Server	Uvicorn	Asynchronous web server running FastAPI on localhost:8000.
ORM & Database	SQLAlchemy 2.0 + PostgreSQL	Relational data persistence for Users, Workspaces, Documents, Concept Graphs, and Chat.
Security & Auth	PyJWT + Passlib (Bcrypt)	Password hashing (bcrypt) and stateless JWT Bearer token authentication.
Parsers	PyPDF + python-docx	Extracting raw plain text from PDF pages and DOCX paragraphs.
AI Model	Google Gemini API (gemini-2.5-flash)	LLM integration for Markdown notes, exam answers, JSON graphs, and active recall grading.

3. High-Level System Architecture & Flow

The application follows a clean 3-tier decoupled architecture:

- 1. **React SPA Frontend** → Communicates via HTTP REST endpoints carrying JWT Bearer authorization.
- 2. **FastAPI Layered Backend** → Request Handlers (Routes) → Business Logic & Algorithms (Services) → Data Schemas (Pydantic/SQLAlchemy).
- 3. **PostgreSQL Database & Google Gemini API** → Dual persistence and intelligence providers.



4. Complete File-by-File Breakdown & Responsibilities

- **backend/app/main.py:** FastAPI app setup, CORS middleware configuration, DB auto-init lifespan handler, router inclusions, and static file mounting (/uploads).
- **backend/app/config.py:** Environment variables loader (DATABASE_URL, JWT_SECRET_KEY, GEMINI_API_KEY), storage paths, and upload size/MIME type constraints.
- **backend/app/database.py:** SQLAlchemy engine, session factory (get_db dependency), Base model, and safe table schema migrations (_safe_add_column).
- **backend/app/models/concept_graph.py:** Database schemas for ConceptNode (title, summary, sub_points JSONB, answer_cache), ConceptEdge (prerequisites), and NodeMastery (LOCKED/UNLOCKED/MASTERED).
- **backend/app/models/generated_content.py:** GeneratedContent table storing AI outputs (REVISION, EXAM, QUIZ, LOGIC_FLOW) with titles and metadata.
- **backend/app/services/graph_service.py:** Core graph algorithms: detect_and_resolve_cycles (DFS DAG builder), generate_concept_graph, evaluate_mastery_states, grade_explanation, and generate_node_answer caching.
- **backend/app/services/content_service.py:** Prompt engineering for Revision Guides, Exam Preparation (with Exam Coaching Boxes), MCQ Quizzes, and Logic Flow step JSON.
- **backend/app/services/chat_service.py:** Workspace-bounded chat service. Merges document context with active concept focus to generate precise AI answers.
- **backend/app/routes/graph.py:** API routes for graph generation (/graph/generate), graph fetch (/graph), node answer (/node/answer), and mastery submission (/attempt).

- **frontend/src/App.jsx**: Root layout component. Manages auth session, dark/light theme, workspace state, browser tab lifecycle, and chat state.
- **frontend/src/components/AnimatedRoadmap.jsx**: Interactive roadmap visualizer. Handles portion text selection, format customization, concept node lock status, and progress bar.
- **frontend/src/components/LogicFlowPanel.jsx**: Step-by-step flowchart simulator with playback controls (Play, Pause, Step Forward/Back, Speed selector).
- **frontend/src/components/TabManager.jsx**: Center panel with browser tabs. Pins Chat Copilot tab, renders document previews, and node reference answers.

5. Viva & Project Review Question & Answer Cheat Sheet

Q1: Why did you choose FastAPI over Flask or Django?

FastAPI is built on modern Python 3.11 asynchronous standards (asyncio) and ASGI (uvicorn), providing performance on par with Node.js and Go. It enforces data validation using Pydantic and automatically generates interactive OpenAPI documentation (/docs).

Q2: How do you handle JWT authentication securely?

Upon user login in `auth_service.py`, a signed JWT token is returned using HS256 encryption. The React client stores the token in local storage and includes it in the Authorization: Bearer header for every request. Routes validate the token using FastAPI's `Depends(get_current_user)` dependency.

Q3: How does the AI prevent context confusion between different subjects?

Through Subject-Isolated Workspaces. Every document, chat message, and concept graph is strictly bound to a `workspace_id` foreign key. When querying Gemini, the backend extracts text exclusively from documents belonging to the active workspace.

Q4: How do you extract text from uploaded PDF and Word files?

When a document is uploaded in `document_service.py`, we validate the file extension. For PDFs, we use `pypdf.PdfReader` to extract text page-by-page. For Word files, `python-docx` parses paragraphs. The raw text is stored in the `ExtractedText` table.

Q5: What happens if Gemini generates a circular dependency in the graph?

In `graph_service.py`, we run a cycle resolution algorithm (`detect_and_resolve_cycles`) using Depth-First Search (DFS). If a cycle (`A -> B -> C -> A`) is detected, the back-edge prerequisite is stripped out, guaranteeing a Directed Acyclic Graph (DAG).

Q6: How does the system ensure generated answers don't hallucinate?

In `content_service.py` and `chat_service.py`, prompts include strict context-bounding instructions: 'Stay strictly within the context of the provided study materials. If the answer cannot be found, state that clearly.'

Q7: How is answer generation optimized for fast response times?

We implement Answer Caching on concept nodes. When a student clicks a concept for the first time, Gemini generates a detailed reference answer which is saved in `ConceptNode.answer_cache`. Subsequent clicks load instantly from PostgreSQL without calling the Gemini API again.